

# **MAJOR PROJECT: MID-TERM REPORT**

On

## **ADAPTIVE RENDERING ENGINE**

**Submitted By**

**Bijay Bk – 220305**

**Devendra Pandey – 220306**

**Manish Joshi – 220312**

**Pramod Panta – 220317**

Under the Supervision of

**Er. Robinhood Khadka**

**Submitted to**

**The Department of ICT and Computer Engineering**

In Partial fulfillment of the requirements for the degree of

**Bachelor of Engineering in Computer**

**Cosmos College of Management & Technology**

(Affiliated to Pokhara University)

Sitapaila, Kathmandu, Nepal

**Date of Submission: Asar 2083 B.S. (June 2026 A.D.)**

## **CERTIFICATE**

The undersigned certify that they have read and recommended to the Department of ICT and Computer Engineering, a mid-term project work entitled “Adaptive Rendering Engine” submitted by Bijay Bk (220305), Devendra Pandey (220306), Manish Joshi (220312) and Pramod Panta (220317) in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer.

---

Er. Robinhood Khadka

(Project Supervisor)

Department of ICT and Computer Engineering

Cosmos College of Management & Technology

---

(External Examiner)

---

Head of the Department

Department of ICT and Computer Engineering

Cosmos College of Management & Technology

## **ACKNOWLEDGEMENT**

We would like to express our sincere gratitude to our project supervisor, Er. Robinhood Khadka, for his continuous guidance, encouragement and valuable feedback throughout the development of the Adaptive Rendering Engine. We are equally thankful to the Department of ICT and Computer Engineering, Cosmos College of Management & Technology, for providing the academic environment and resources required to carry out this work.

We also acknowledge the open-source community behind Node.js, React, Docker and the wider web-performance ecosystem, whose tools made it possible to build and evaluate this project entirely at zero cost. Finally, we thank our families and friends for their constant support.

Bijay Bk (220305)

Devendra Pandey (220306)

Manish Joshi (220312)

Pramod Panta (220317)

# ABSTRACT

Modern web applications employ multiple rendering strategies—Static Site Generation (SSG), Server-Side Rendering (SSR), Client-Side Rendering (CSR), Incremental Static Regeneration (ISR), Streaming SSR and Edge rendering—to balance performance, scalability and data freshness. In existing frameworks such as Next.js and Remix this strategy is fixed by the developer at build time and applied uniformly to every request, which does not adapt to changing runtime conditions such as network speed, device capability, server load or data volatility, often producing sub-optimal performance.

This project designs and implements an Adaptive Rendering Engine (ARE)—a runtime system that automatically selects the optimal rendering strategy on a per-request basis using a pure, rule-based decision engine that analyses contextual factors. The engine is built with open-source technologies (Node.js, TypeScript and React 18 on a native HTTP server) and is evaluated on a zero-cost, Docker-based private server that models an origin and multiple edge nodes.

This mid-term report documents the progress achieved to date. The core runtime, all six rendering strategy modules, the decision engine, the caching layer and the metrics subsystem have been implemented and verified locally: the same URL is demonstrably served with six different strategies depending on request context, all nine decision rules pass automated unit tests, and the system has been containerised successfully. Preliminary performance measurements confirm the expected trade-offs between strategies. The remaining work—full multi-node experimentation, comprehensive benchmarking and statistical analysis—is scheduled for the second half of the project.

# TABLE OF CONTENTS

|                                                       |             |
|-------------------------------------------------------|-------------|
| <b>CERTIFICATE.....</b>                               | <b>i</b>    |
| <b>ACKNOWLEDGEMENT.....</b>                           | <b>ii</b>   |
| <b>ABSTRACT.....</b>                                  | <b>iii</b>  |
| <b>LIST OF FIGURES.....</b>                           | <b>vi</b>   |
| <b>LIST OF TABLES.....</b>                            | <b>vii</b>  |
| <b>LIST OF ACRONYMS / ABBREVIATIONS.....</b>          | <b>viii</b> |
| <b>1. Introduction.....</b>                           | <b>1</b>    |
| 1.1 Introduction to the Project.....                  | 1           |
| 1.2 Background.....                                   | 1           |
| 1.3 Problem Statement.....                            | 1           |
| 1.4 Objectives.....                                   | 1           |
| 1.5 Scope of the Project.....                         | 2           |
| 1.6 Team Members and Plan Followed.....               | 2           |
| 1.7 Application of the Project.....                   | 3           |
| <b>2. Literature Review.....</b>                      | <b>4</b>    |
| <b>3. Methodology and System Development.....</b>     | <b>5</b>    |
| 3.1 Research Design.....                              | 5           |
| 3.2 Technology Stack.....                             | 5           |
| 3.3 System Architecture.....                          | 6           |
| 3.4 Decision Algorithm.....                           | 6           |
| 3.5 Rendering Pipeline.....                           | 7           |
| 3.6 Rendering Strategy Modules.....                   | 8           |
| 3.7 Caching and Metrics.....                          | 8           |
| 3.8 Validation Method.....                            | 8           |
| <b>4. Progress Summary.....</b>                       | <b>9</b>    |
| <b>5. Preliminary Results.....</b>                    | <b>10</b>   |
| 5.1 Functional Verification – Strategy Switching..... | 10          |
| 5.2 Unit Testing.....                                 | 10          |
| 5.3 Preliminary Performance Measurements.....         | 10          |

|                                               |           |
|-----------------------------------------------|-----------|
| 5.4 Discussion.....                           | 11        |
| <b>6. Challenges Faced and Solutions.....</b> | <b>12</b> |
| <b>7. Updated Plan and Timeline.....</b>      | <b>13</b> |
| <b>8. Conclusion.....</b>                     | <b>14</b> |
| <b>9. References.....</b>                     | <b>15</b> |
| <b>Appendices.....</b>                        | <b>16</b> |

## LIST OF FIGURES

Figure 3.1: High-Level System Architecture of the ARE Private Server

Figure 3.2: Decision Engine Rule Flow (first match wins)

Figure 3.3: Per-Request Rendering Pipeline

## **LIST OF TABLES**

Table 1.1: Project Team Members

Table 3.1: Technology Stack and Justification

Table 3.2: Decision Rule Table (Context → Strategy)

Table 4.1: Progress Against the Initial Timeline

Table 5.1: Strategy-Switching Verification (Same URL, Varied Context)

Table 5.2: Preliminary Per-Strategy Performance Measurements

Table 6.1: Challenges Faced and Solutions Adopted

Table 7.1: Updated Plan – Remaining Tasks and Timeline

## LIST OF ACRONYMS / ABBREVIATIONS

|             |                                   |
|-------------|-----------------------------------|
| <b>ARE</b>  | Adaptive Rendering Engine         |
| <b>SSG</b>  | Static Site Generation            |
| <b>SSR</b>  | Server-Side Rendering             |
| <b>CSR</b>  | Client-Side Rendering             |
| <b>ISR</b>  | Incremental Static Regeneration   |
| <b>TTFB</b> | Time To First Byte                |
| <b>FCP</b>  | First Contentful Paint            |
| <b>LCP</b>  | Largest Contentful Paint          |
| <b>HTTP</b> | HyperText Transfer Protocol       |
| <b>HTML</b> | HyperText Markup Language         |
| <b>CDN</b>  | Content Delivery Network          |
| <b>TS</b>   | TypeScript                        |
| <b>TTL</b>  | Time To Live                      |
| <b>API</b>  | Application Programming Interface |
| <b>SEO</b>  | Search Engine Optimization        |

# 1. Introduction

## 1.1 Introduction to the Project

The Adaptive Rendering Engine (ARE) is a runtime system that automatically selects the most appropriate web rendering strategy for each incoming request. Rather than fixing a single rendering method at development time, the ARE analyses the live context of every request and chooses among six strategies—SSG, SSR, Streaming SSR, ISR, CSR and a simulated Edge-ISR—to optimise response time, resource usage and user experience. The project is built as technology (an engine), not as an end-user application, and runs entirely on open-source tools with no commercial cloud dependency.

## 1.2 Background

Rendering strategies such as SSR, CSR, SSG, ISR and Streaming SSR have been widely studied and implemented to improve performance and scalability [2], [5]. Each offers distinct advantages and trade-offs in latency, scalability, SEO and user experience [3], [4]. However, in current frameworks the choice is static and developer-defined, and therefore cannot react to network fluctuations, server-load spikes, device heterogeneity or content volatility at runtime [7], [11].

## 1.3 Problem Statement

*Rendering strategy selection in modern web applications is static and does not adapt to runtime contextual conditions, resulting in reduced performance efficiency and scalability limitations. There is currently no unified runtime engine that combines contextual analysis with rendering orchestration at the application layer [2], [5], [11].*

## 1.4 Objectives

### 1.4.1 General Objective

To design, implement and evaluate an Adaptive Rendering Engine that dynamically selects rendering strategies at runtime based on contextual variables in order to optimise web performance.

### 1.4.2 Specific Objectives

1. To identify and define the contextual variables influencing rendering performance (network condition, server load, device type, cache state, data volatility).
2. To design a pure, rule-based decision engine that maps contextual inputs to an optimal rendering strategy.
3. To implement modular rendering strategies (SSG, SSR, Streaming SSR, ISR, CSR and simulated Edge-ISR) behind a single interface.
4. To measure performance metrics such as TTFB, render time, response size, cache efficiency and resource usage.
5. To experimentally compare adaptive rendering against static rendering configurations on a Docker-based private server.

### 1.5 Scope of the Project

The project covers the design and implementation of a standalone Adaptive Rendering Engine, the simulation of network conditions, device profiles and server load within a local Docker-based environment, and the comparative evaluation of adaptive versus static rendering. It does not include deployment to commercial cloud infrastructure, development of a full production-scale web application, or machine-learning-based optimisation, which is identified as future work.

### 1.6 Team Members and Plan Followed

*Table 1.1: Project Team Members*

| Name            | Roll No. | Primary Responsibility              |
|-----------------|----------|-------------------------------------|
| Bijay Bk        | 220305   | Core engine, decision logic, server |
| Devendra Pandey | 220306   | Rendering strategy modules          |
| Manish Joshi    | 220312   | Caching, metrics, Docker setup      |
| Pramod Panta    | 220317   | Testing, validation, documentation  |

The team follows an incremental, milestone-driven plan with weekly checkpoints, version control and regular supervisor reviews, as detailed in Chapter 7.

## **1.7 Application of the Project**

- Framework-level optimisation: the engine can be integrated as a middleware layer to optimise rendering per request.
- Low-bandwidth environments: educational or government sites can serve lighter strategies on slow networks.
- High-traffic scenarios: cached strategies reduce origin load during spikes.
- Device-aware rendering: low-end devices receive minimal JavaScript while capable devices get richer interactivity.
- Research and education: a platform for studying rendering trade-offs.

## 2. Literature Review

Comparative studies show that SSR improves initial page-load performance and SEO because content is rendered before reaching the browser, but increases server workload under high traffic [4], [6]. CSR shifts rendering to the client, reducing server overhead but often delaying First Contentful Paint under weak networks [3], [9]. Hybrid approaches such as SSG and ISR pre-generate content to reduce server strain and improve caching, at the cost of content freshness [5].

Web-performance research emphasises measurable indicators such as TTFB, FCP, LCP, CPU utilisation, memory consumption and cache-hit ratio [7], [8], while mobile and edge-caching studies highlight the value of contextual adaptation in reducing latency [9], [10], [11]. These works, however, evaluate rendering strategies in isolation and apply contextual adaptation at the network or infrastructure layer rather than at the rendering-selection level.

The identified research gap is the absence of a unified runtime engine that integrates contextual analysis with rendering orchestration and is experimentally validated under controlled conditions. This project directly addresses that gap [1], [2], [5], [11], [12], [13].

### 3. Methodology and System Development

#### 3.1 Research Design

The project adopts a design-and-build methodology combined with a quantitative experimental evaluation. The independent variables are the contextual factors (network speed, server load, device type, cache state, data volatility); the moderating variable is the selected rendering strategy; and the dependent variables are TTFB, render time, response size, cache-hit ratio and resource usage. Controlled simulations isolate the impact of strategy selection under varying runtime conditions.

#### 3.2 Technology Stack

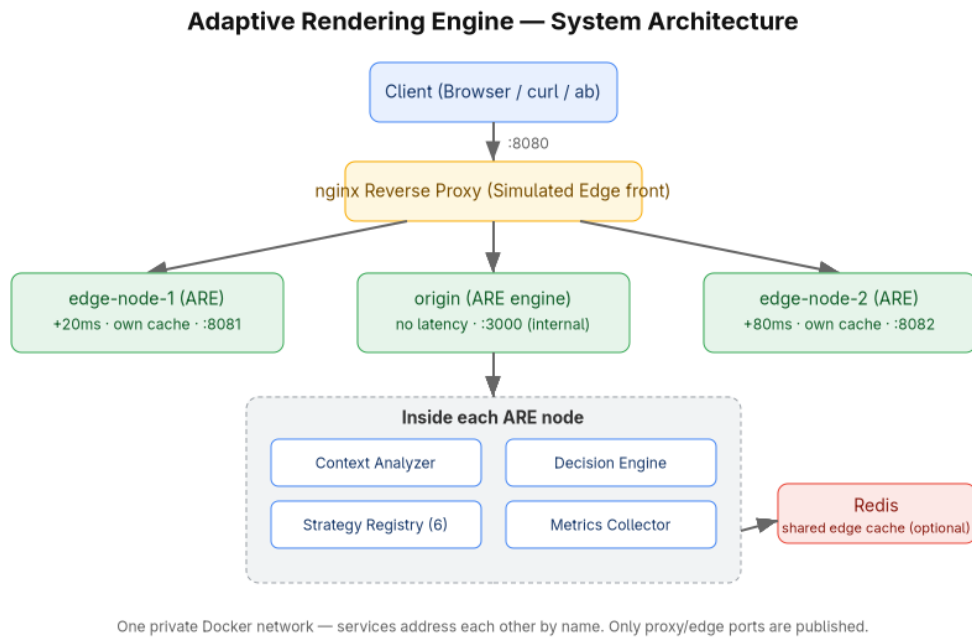
The stack is pinned to free, open-source, industry-standard tools. React 18 is used purely as the rendering primitive—the contribution is the selector engine built around it, not a new view library.

**Table 3.1: Technology Stack and Justification**

| Concern       | Technology                  | Reason                                          |
|---------------|-----------------------------|-------------------------------------------------|
| Language      | Node.js 20+ / TypeScript    | Type-safe engine; one language end-to-end       |
| View layer    | React 18                    | Supports SSR, streaming and hydration natively  |
| HTTP server   | Native Node http            | The engine is the runtime; no framework lock-in |
| Client bundle | esbuild                     | Fast, zero-config CSR/hydration bundle          |
| Caching       | Filesystem + memory + Redis | Zero-cost persistent and shared caching         |
| Edge / proxy  | nginx + Docker Compose      | Models origin and edge nodes locally            |
| Testing       | Vitest                      | Fast, TypeScript-native unit tests              |
| Load testing  | Apache Bench (ab)           | Concurrent-request stress testing               |

### 3.3 System Architecture

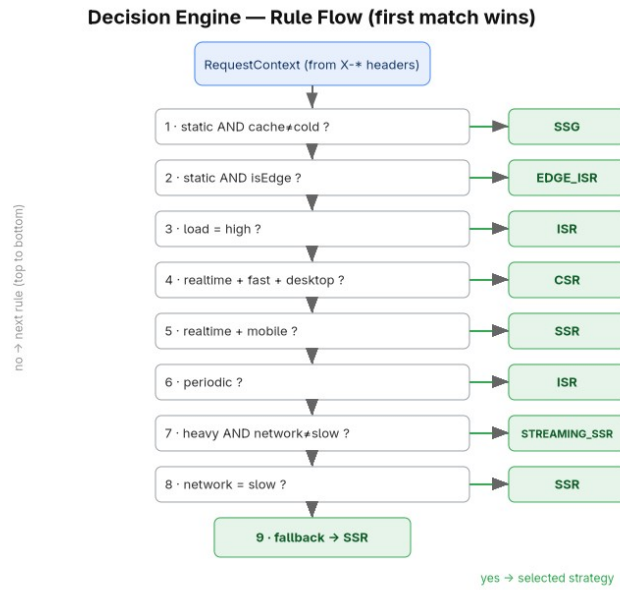
The ARE is deployed as a Docker-based private server consisting of an origin node running the engine, two nginx-fronted edge nodes with injected latency, an optional shared Redis cache and a reverse proxy. Containers share one private network and address each other by name, so no public IP or paid cloud is required.



**Figure 3.1: High-Level System Architecture of the ARE Private Server**

### 3.4 Decision Algorithm

The decision engine is a pure, deterministic, input-output function that evaluates a rule table top-to-bottom; the first matching rule wins, and an unconditional fallback guarantees a result. Because it performs no I/O, it is fully unit-testable.



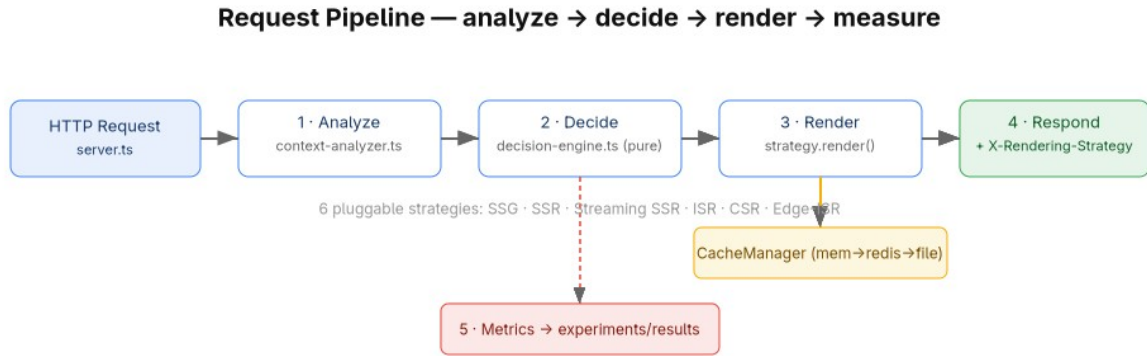
**Figure 3.2: Decision Engine Rule Flow (first match wins)**

**Table 3.2: Decision Rule Table (Context → Strategy)**

| # | Condition (first match wins)             | Strategy      |
|---|------------------------------------------|---------------|
| 1 | static AND cache ≠ cold                  | SSG           |
| 2 | static AND at edge                       | EDGE_ISR      |
| 3 | load = high                              | ISR           |
| 4 | realtime AND fast network<br>AND desktop | CSR           |
| 5 | realtime AND mobile                      | SSR           |
| 6 | periodic data                            | ISR           |
| 7 | heavy payload AND network ≠<br>slow      | STREAMING_SSR |
| 8 | network = slow                           | SSR           |
| 9 | fallback (no rule matched)               | SSR           |

### 3.5 Rendering Pipeline

Each request flows through a four-stage pipeline: analyse (build the request context from headers, with inference fallbacks), decide (run the rule table), render (delegate to the chosen strategy), and respond and measure (set the X-Rendering-Strategy header, log the decision and record metrics).



**Figure 3.3: Per-Request Rendering Pipeline**

### 3.6 Rendering Strategy Modules

All six strategies implement a single `RenderStrategy` interface and self-register, so the engine can select any of them uniformly: SSG serves pre-built HTML; SSR renders fresh HTML per request; Streaming SSR streams the shell first and Suspense boundaries as they resolve; ISR serves cached content with stale-while-revalidate; CSR sends a minimal shell that hydrates on the client; and Edge-ISR applies ISR semantics with an edge-namespaced, optionally Redis-shared cache.

### 3.7 Caching and Metrics

A three-tier cache (memory → Redis → filesystem) backs ISR and Edge-ISR and enables cold-versus-warm comparisons. A metrics collector records per-request TTFB, render time, response size, cache-hit status and resource usage as newline-delimited JSON, which a report generator aggregates into per-strategy tables.

### 3.8 Validation Method

Functional correctness is validated by issuing multiple HTTP requests to the same endpoint under varying contextual headers and observing the selected strategy, which is returned in the `X-Rendering-Strategy` response header and written to the server log. This provides reproducible, inspectable evidence of runtime strategy switching.

## 4. Progress Summary

As of this mid-term submission, the project has reached the stage at which the complete engine runs and is verifiable locally. The following components are implemented and tested:

- Core runtime: context analyzer, pure decision engine, strategy registry and the request orchestrator.
- All six rendering strategy modules (SSG, SSR, Streaming SSR, ISR, CSR, Edge-ISR) behind one interface.
- Three-tier caching layer with stale-while-revalidate revalidation.
- Metrics collection and an aggregating performance-report generator.
- React 18 test pages and an esbuild-built client bundle for hydration and CSR.
- A multi-stage Docker image that builds and runs successfully, plus a docker-compose topology for origin, edges, proxy and Redis.
- Automated validation scripts for strategy switching, header inspection and load testing.

The build compiles cleanly under the TypeScript type-checker, all automated unit tests pass, and the containerised image boots and serves requests correctly. Evidence is presented in Chapter 5.

**Table 4.1: Progress Against the Initial Timeline**

| Phase | Planned Activity                                | Status                                                                      |
|-------|-------------------------------------------------|-----------------------------------------------------------------------------|
| 1     | Research consolidation & system design          | Completed                                                                   |
| 2     | Core engine & rendering strategy development    | Completed                                                                   |
| 3     | Deployment & environment configuration (Docker) | Largely complete – image builds & runs; full multi-node compose run pending |
| 4     | Experimental testing & data collection          | In progress – preliminary metrics collected                                 |
| 5     | Data analysis & final documentation             | Pending (next half)                                                         |

## 5. Preliminary Results

### 5.1 Functional Verification – Strategy Switching

The core hypothesis was verified directly: issuing requests to the same URLs while varying only the contextual headers caused the engine to select six different strategies, each matching the expected rule outcome. Table 5.1 reproduces the output of the automated switch-test script.

**Table 5.1: Strategy-Switching Verification (Same URL, Varied Context)**

| Request context (headers)                 | Selected      | Expected      |
|-------------------------------------------|---------------|---------------|
| static volatility, /static                | SSG           | SSG           |
| realtime + fast + desktop,<br>/dynamic    | CSR           | CSR           |
| high load, /dynamic                       | ISR           | ISR           |
| realtime + mobile, /dynamic               | SSR           | SSR           |
| heavy payload + medium network,<br>/heavy | STREAMING_SSR | STREAMING_SSR |
| static + edge + cold cache,<br>/dynamic   | EDGE_ISR      | EDGE_ISR      |
| periodic, /dynamic                        | ISR           | ISR           |

### 5.2 Unit Testing

The decision engine is covered by automated tests with one assertion per rule row. All 14 tests across the decision-engine, cache and rendering suites pass, demonstrating that strategy selection is deterministic and correct rather than incidental.

### 5.3 Preliminary Performance Measurements

Table 5.2 summarises per-strategy measurements aggregated by the report generator during local runs. The numbers confirm the expected trade-offs: CSR ships the smallest payload (a ~0.4 KB shell), Streaming SSR carries the largest (a heavy ~20 KB page), the cache-backed strategies (SSG, ISR, Edge-ISR) achieve a cache-hit rate of 1.0 with very low TTFB, while SSR and Streaming SSR incur a higher server-side render cost. These are indicative figures from a development machine; the full controlled benchmark is part of the remaining work.

**Table 5.2: Preliminary Per-Strategy Performance Measurements**

| Strategy      | Avg TTFB (ms) | Avg Render (ms) | Cache-hit | Avg Bytes |
|---------------|---------------|-----------------|-----------|-----------|
| SSG           | 1.24          | 0.74            | 1.0       | 1,199     |
| CSR           | 0.31          | 0.16            | 0.0       | 401       |
| ISR           | 0.41          | 0.26            | 1.0       | 1,219     |
| SSR           | 6.45          | 0.16            | 0.0       | 1,219     |
| Streaming SSR | 8.36          | 0.79            | 0.0       | 20,896    |
| Edge-ISR      | 1.48          | 0.20            | 1.0       | 1,234     |

## 5.4 Discussion

The preliminary results validate the design intent. The engine demonstrably adapts to context, the caching strategies reduce TTFB relative to per-request rendering, and the payload differences confirm that the engine tailors the response weight to the situation (a light shell for capable clients, progressive streaming for heavy pages). The next phase will repeat these measurements under the full Docker multi-node setup with network throttling and concurrent load to produce statistically robust comparisons against fixed-strategy baselines.

## 6. Challenges Faced and Solutions

*Table 6.1: Challenges Faced and Solutions Adopted*

| Challenge                                                   | Impact | Solution Adopted                                                                                                                         |
|-------------------------------------------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------|
| Non-ASCII characters in HTTP response headers caused errors | Medium | Sanitised the decision-reason header to ASCII while keeping full text in logs                                                            |
| nginx cannot natively inject latency for edge emulation     | Medium | Modelled edges as real ARE service replicas with configurable latency and self-identification, giving each edge a real process and cache |
| Streaming SSR must not be buffered                          | Medium | Returned a Node stream piped directly to the response so chunks flush as produced                                                        |
| Chicken-and-egg: SSG selectable only when cache is warm     | Low    | Server pre-builds static pages on startup so SSG is selectable from the first request                                                    |
| Docker Compose plugin not present on the host               | Low    | Verified the image via direct docker build/run; documented the one-line plugin install                                                   |

## 7. Updated Plan and Timeline

Phases 1 and 2 are complete and Phase 3 is largely complete, ahead of or on schedule. The remaining work concentrates on full deployment, experimentation and analysis, summarised below.

**Table 7.1: Updated Plan – Remaining Tasks and Timeline**

| Remaining Task                                                              | Planned Window |
|-----------------------------------------------------------------------------|----------------|
| Install Docker Compose and run the full origin + edge + proxy + Redis stack | Week 9–10      |
| Network-condition and device-profile experiments (2G/3G/4G, mobile/desktop) | Week 11–12     |
| Load testing with Apache Bench and cold-vs-warm cache experiments           | Week 12–13     |
| Adaptive-vs-static comparative benchmarking and statistical analysis        | Week 13–14     |
| Graph generation, results interpretation and final report                   | Week 14–16     |

## **8. Conclusion**

This mid-term report has presented the progress of the Adaptive Rendering Engine. The project has successfully moved from design to a working, locally verifiable runtime: the engine analyses request context, selects one of six rendering strategies through a pure rule-based decision engine, renders with React 18 and reports detailed metrics, all on a zero-cost open-source stack. Functional correctness has been demonstrated through live strategy switching on identical URLs and through passing unit tests, and preliminary measurements confirm the expected performance trade-offs. The foundation is therefore solid, and the remaining effort—full multi-node experimentation and comparative analysis—is well defined and on schedule for the final submission.

## 9. References

- [1] Q. Wang, H. Shen, Y. Li, and Z. Zhang, “Anatomizing Deep Learning Inference in Web Browsers,” in *Proc. Web Conf. 2021 (WWW ’21)*, Ljubljana, Slovenia, 2021, pp. 2819–2830, doi: 10.1145/3442381.3449890.
- [2] M. Alshammari and R. Alshammari, “Comparison of Web Page Rendering Methods,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 13, no. 4, pp. 95–103, 2022.
- [3] A. Kumar and S. Patel, “Server-Side vs Client-Side Rendering: Performance Evaluation Study,” *Int. J. Comput. Appl.*, vol. 183, no. 12, pp. 1–8, 2021.
- [4] J. Lee and K. Park, “The Impact of Server-Side Rendering on UI Performance and SEO,” *J. Web Eng.*, vol. 21, no. 6, pp. 1503–1522, 2022.
- [5] S. Ibrahim, T. Hassan, and M. Noor, “Web Rendering Strategies: A Comparative Analysis,” *IEEE Access*, vol. 11, pp. 45678–45692, 2023.
- [6] R. Singh and P. Verma, “Experimental Analysis of Server-Side Caching for Web Performance,” *ACM Trans. Web*, vol. 14, no. 3, pp. 1–25, 2020.
- [7] L. Chen, Y. Zhao, and M. Li, “Overview of Web Application Performance Optimization Techniques,” *J. Syst. Softw.*, vol. 180, 111002, 2021.
- [8] H. Rahman and D. Kim, “Frontend Performance Optimization in Modern Web Applications,” in *Proc. Int. Conf. Software Engineering Advances*, 2020, pp. 87–94.
- [9] M. A. Hassan, S. Islam, and R. Rahman, “Improving Perceived Performance of Mobile Web Applications,” *Mobile Inf. Syst.*, vol. 2019, Art. ID 7483956, 2019.
- [10] N. Alotaibi and A. Alharbi, “Optimizing Web Interface Rendering for Mobile Applications with High User Traffic,” *Int. J. Comput. Appl.*, vol. 185, no. 7, pp. 95–103, 2023.
- [11] Y. Zhang, X. Wang, and J. Liu, “Mobility-Aware Edge Caching for Minimizing Latency in Vehicular Networks,” *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8321–8334, 2021.
- [12] A. R. Mahmoud, “Rendering Strategies and User Experience Analysis,” M.S. thesis, Dept. Comput. Sci., Univ. of Technology, 2022.
- [13] S. Khalid and M. Usman, “Technologies for Modern Web Application Architecture,” *J. Distrib. Syst. Web Technol.*, vol. 5, no. 2, pp. 44–58, 2023.

## Appendices

### Appendix A: Sample Strategy-Switch Log

The engine logs every decision in the form below, providing direct evidence of runtime selection:

[ARE] Request: /dynamic

[ARE] Context: net=fast device=desktop cache=fresh load=low volatility=realtime heavy=false edge=false

[ARE] Strategy selected: CSR – Realtime data on a capable client -> fully interactive (CSR)

### Appendix B: Repository Structure (Top Level)

src/ (core, strategies, cache, metrics, simulation, server, frontend, config, utils), docker/ and docker-compose.yml, scripts/ (validation), tests/, experiments/ (results), docs/ and diagrams/. Full design documents accompany the source code.